In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels as sm
import scipy as sp
```

In [3]:
```
df = pd.read_csv('downloads/marketing.csv')
```

In [4]:
```
df.head()
```

Out[4]:

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.518070	2.293790	Micro	55.261284
1	Low	7.756876	2.572287	Mega	67.574904
2	High	20.348988	1.227180	Micro	272.250108
3	Medium	20.108487	2.728374	Mega	195.102176
4	High	31.653200	7.776978	Nano	273.960377

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   TV            572 non-null    object 
 1   Radio         572 non-null    float64
 2   Social Media  572 non-null    float64
 3   Influencer    572 non-null    object 
 4   Sales         572 non-null    float64
dtypes: float64(3), object(2)
memory usage: 22.5+ KB
```

In [6]:
```
df.shape
```

Out[6]: (572, 5)

In [7]:
```
df.duplicated().sum()
```

Out[7]: np.int64(0)

In [8]:
```
df.describe()
```

Out[8]:

	Radio	Social Media	Sales
count	572.000000	572.000000	572.000000
mean	17.520616	3.333803	189.296908
std	9.290933	2.238378	89.871581
min	0.109106	0.000031	33.509810
25%	10.699556	1.585549	118.718722
50%	17.149517	3.150111	184.005362
75%	24.606396	4.730408	264.500118
max	42.271579	11.403625	357.788195

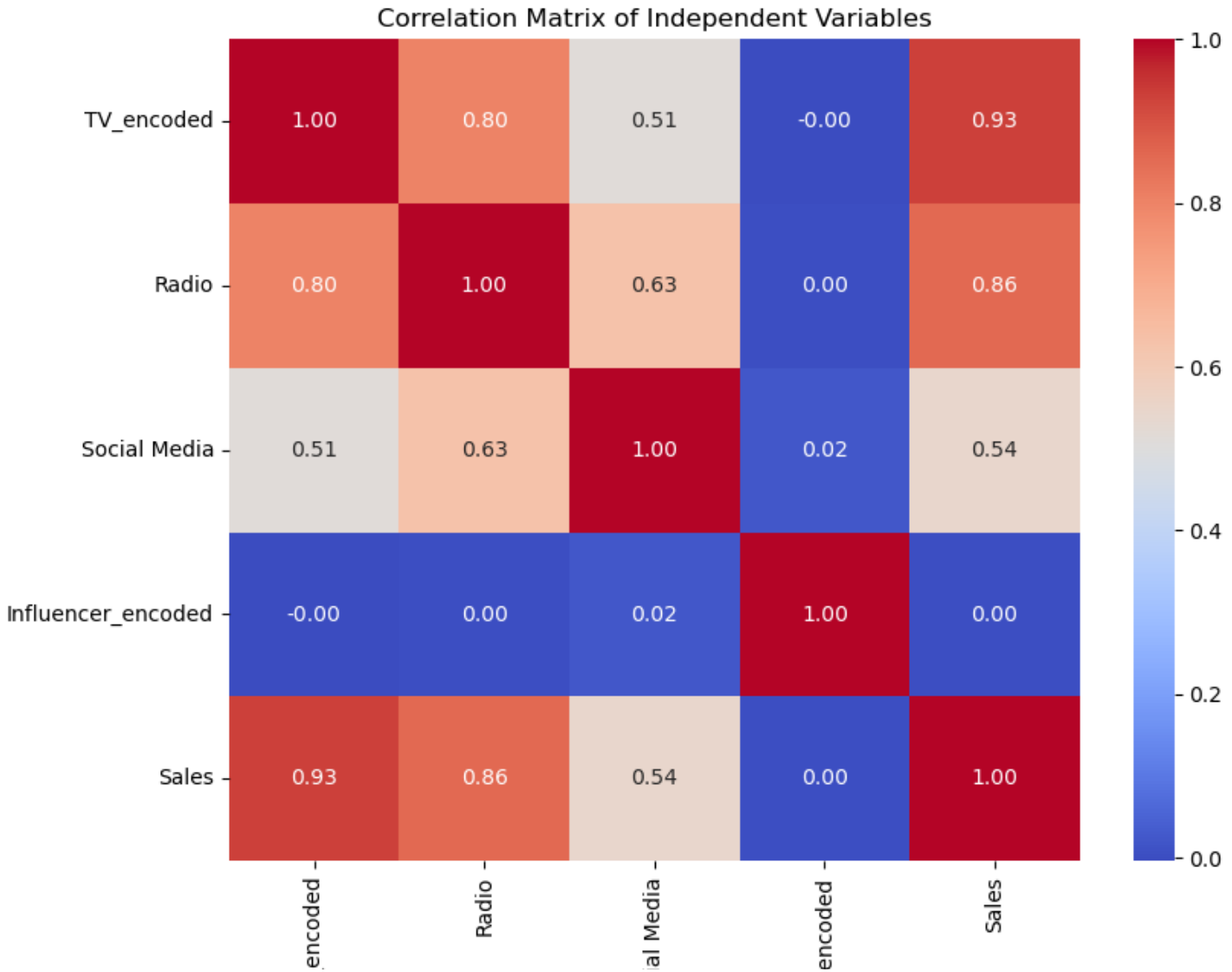
In [11]:
```
for col in df.select_dtypes(include='object').columns:
    print(f"Uniques values for {col}:")
    print(df[col].unique())
    print("/n")
```

```
Uniques values for TV:
['Low' 'High' 'Medium']
/n
Uniques values for Influencer:
['Micro' 'Mega' 'Nano' 'Macro']
/n
```

In [13]:
```
tv_mapping = {'Low':0, 'Medium':1, 'High':2}
df['TV_encoded'] = df['TV'].map(tv_mapping)

influencer_mapping = {'Mega':3, 'Macro':2, 'Micro':1, 'Nano':0}
df['Influencer_encoded'] = df['Influencer'].map(influencer_mapping)

display(df.head())
```

	TV	Radio	Social Media	Influencer	Sales	TV_encoded	Influencer_encoded
0	Low	3.518070	2.293790	Micro	55.261284	0	1
1	Low	7.756876	2.572287	Mega	67.574904	0	3
2	High	20.348988	1.227180	Micro	272.250108	2	1
3	Medium	20.108487	2.728374	Mega	195.102176	1	3
4	High	31.653200	7.776978	Nano	273.960377	2	0

In [14]:
```
vars_for_corr = ['TV_encoded', 'Radio', 'Social Media','Influencer_encoded','Sales']
correlation_matrix = df[vars_for_corr].corr()

plt.figure(figsize=(9,7))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Independent Variables')
plt.show()
```



TV_ Soci Influencer_

In [16]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[['TV_encoded', 'Radio', 'Social Media', 'Influencer_encoded']]
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values,i) for i in range(len(X.columns))]

display(vif_data)
```

	feature	VIF
0	TV_encoded	6.390793
1	Radio	11.895653
2	Social Media	5.243382
3	Influencer_encoded	1.987738

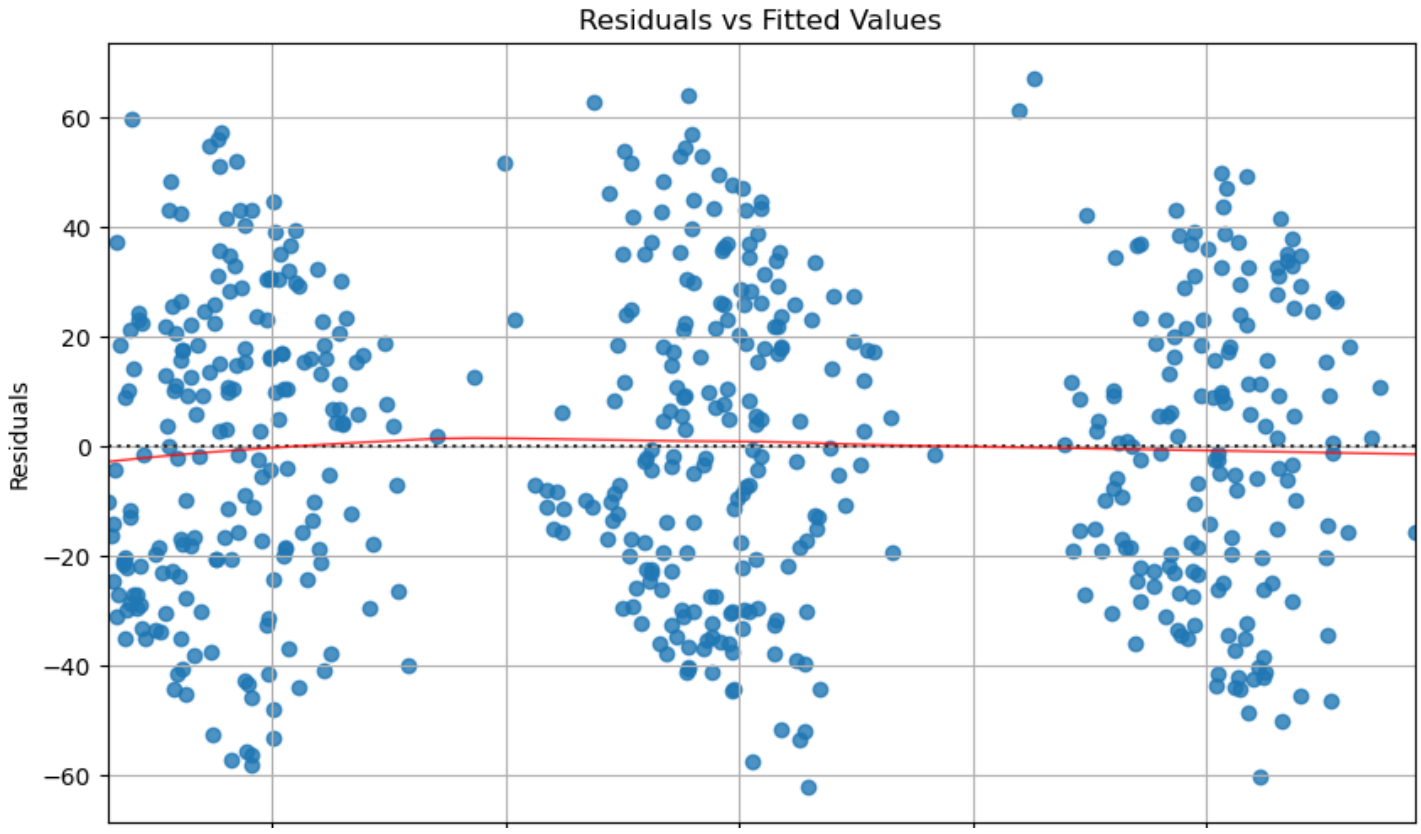
In [18]:
```
import statsmodels.formula.api as smf

model_formula = 'Sales ~ TV_encoded + Radio + Q("Social Media") + Influencer_encoded'

model = smf.ols(model_formula, data=df).fit()

print(model.summary())
```

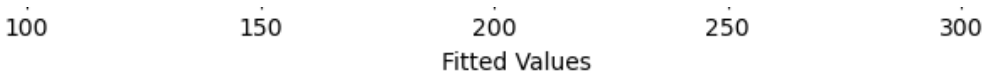
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     1334.
Date:                Sun, 14 Jun 2026   Prob (F-statistic):          9.28e-287
Time:                        12:41:06   Log-Likelihood:                -2714.2
No. Observations:                 572   AIC:                             5438.
Df Residuals:                     567   BIC:                             5460.
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept            64.6348      3.004     21.518      0.000      58.735      70.535
TV_encoded           77.3279      2.458     31.462      0.000      72.500      82.155
Radio                 2.9797      0.234     12.738      0.000       2.520       3.439
Q("Social Media")    -0.1631      0.673     -0.242      0.809      -1.486       1.159
Influencer_encoded    0.2834      1.037      0.273      0.785      -1.753       2.319
==============================================================================
Omnibus:                       59.057   Durbin-Watson:                   1.874
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.812
Skew:                           0.053   Prob(JB):                     0.000136
Kurtosis:                       2.142   Cond. No.                         54.7
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [21]:
```
import pandas as pd
import statsmodels.formula.api as smf

# Load the dataset
df = pd.read_csv('marketing.csv')

# Map 'TV' categories to numerical values
tv_mapping = {'Low': 0, 'Medium': 1, 'High': 2}
df['TV_encoded'] = df['TV'].map(tv_mapping)

print('Adjusted R-squared:', model.rsquared_adj)
print('\nP-values for predictors:')
print(model.pvalues.drop('Intercept'))
```

```
Adjusted R-squared: 0.9032572080713607

P-values for predictors:
TV_encoded            1.832034e-126
Radio                  7.314556e-33
Q("Social Media")      8.086433e-01
Influencer_encoded     7.846513e-01
dtype: float64
```

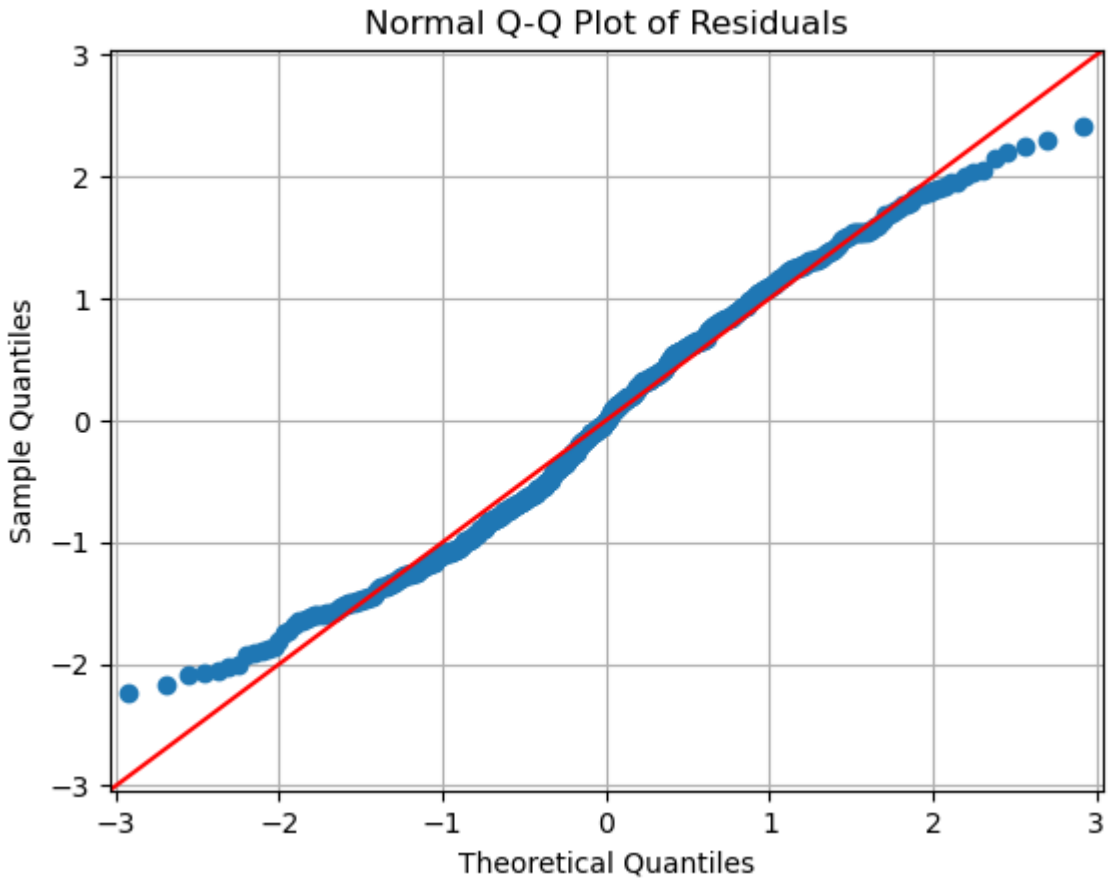
In [22]:
```
import statsmodels.api as sm

# Get residuals and fitted values
residuals = model.resid
fitted_values = model.fittedvalues

# Plot 1: Residuals vs Fitted Values
plt.figure(figsize=(10, 6))
sns.residplot(x=fitted_values, y=residuals, lowess=True, line_kws={'color': 'red', 'lw': 1, 'alpha':
0.8})
plt.title('Residuals vs Fitted Values')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.grid(True)
plt.show()
```



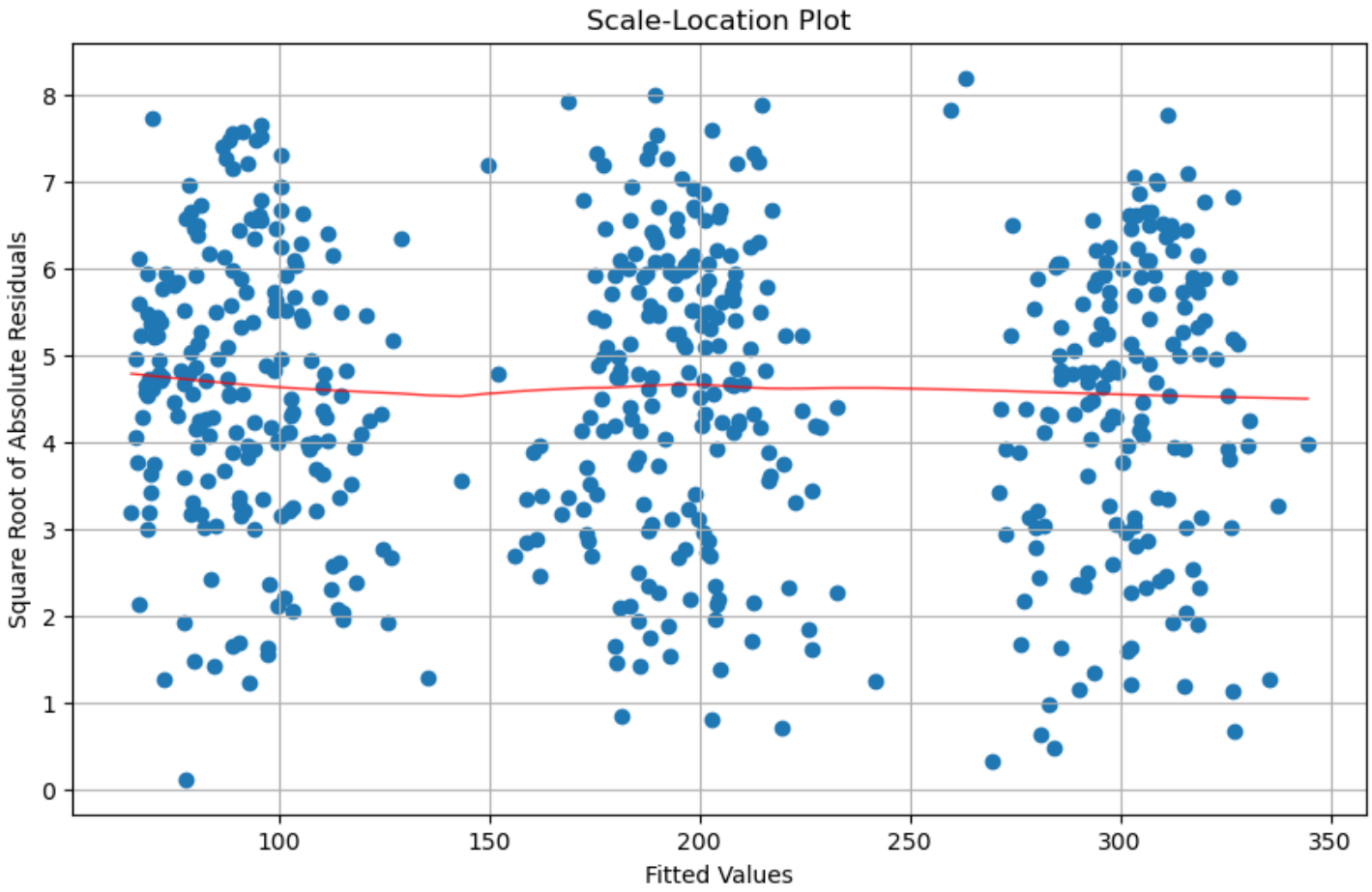


In [23]:
```
# Plot 2: Normal Q-Q Plot
plt.figure(figsize=(10, 6))
sm.qqplot(residuals, line='45', fit=True)
plt.title('Normal Q-Q Plot of Residuals')
plt.grid(True)
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



In [25]:
```
# Plot 3: Scale-Location Plot
plt.figure(figsize=(10, 6))
plt.scatter(x=fitted_values, y=np.sqrt(np.abs(residuals)))
sns.regplot(x=fitted_values, y=np.sqrt(np.abs(residuals)), scatter=False, lowess=True, line_kws={'color':
'red', 'lw': 1, 'alpha': 0.8})
plt.title('Scale-Location Plot')
plt.xlabel('Fitted Values')
plt.ylabel('Square Root of Absolute Residuals')
plt.grid(True)
plt.show()
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:
